

Apéndice D

Documentación técnica de programación

D.1. Introducción

Este apéndice documenta el sistema IUCE Reservas desde la perspectiva del desarrollo, y está dirigido a quien tenga que mantener o evolucionar el código. Recoge la organización del proyecto, el manual del programador, las instrucciones para compilar, instalar y ejecutar el sistema, y la estrategia de pruebas.

El proyecto es una aplicación full-stack en un único repositorio, escrita íntegramente en TypeScript sobre Next.js 14 con el App Router. Esta unificación (frontend, lógica de servidor y acceso a datos en el mismo lenguaje y proyecto) es la decisión estructural que más condiciona la organización del código: no hay un backend separado, sino rutas y componentes que conviven bajo `src/`, y un esquema de datos (`prisma/schema.prisma`) que actúa como fuente única de verdad de la que se derivan tanto la sincronización del esquema con la base de datos (flujo `prisma db push`) como los tipos que usan el resto de las capas.

D.2. Estructura de directorios

El proyecto se organiza en torno a la carpeta `src/`, donde residen la aplicación (`app/`), la lógica de negocio (`lib/`) y los componentes de interfaz (`components/`). El resto de carpetas de la raíz contienen el modelo de datos, los recursos estáticos, la documentación y la configuración de las herramientas.

Estructura del proyecto · iuce-reservas-tfg/

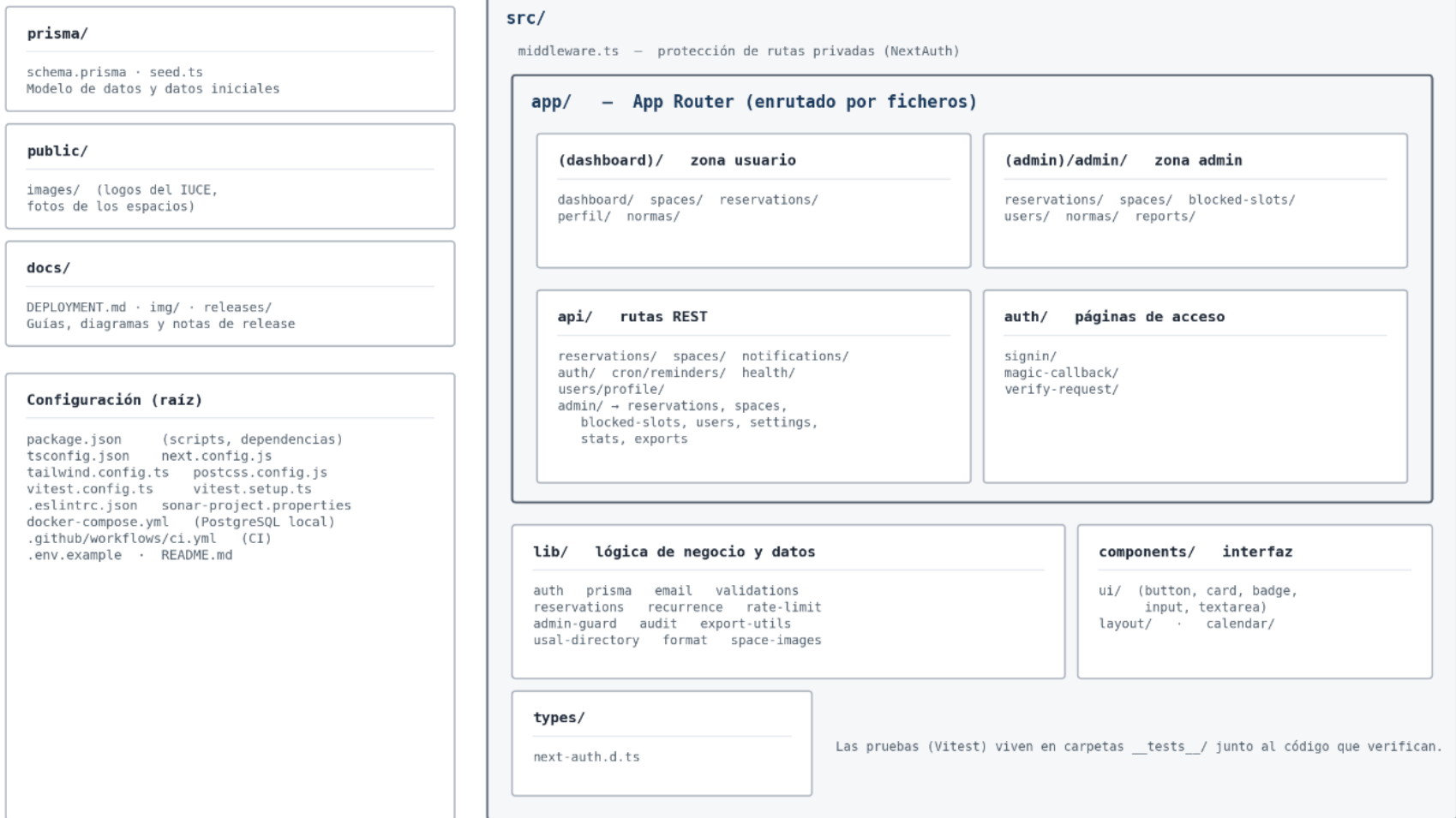


Figura 22. Estructura del proyecto.

D.3. Manual del programador

Este apartado documenta la API REST del backend de IUCE Reservas, implementada sobre el App Router de Next.js 14. Se recogen las convenciones generales, el mecanismo de autenticación, el catálogo completo de endpoints agrupados por área funcional (con su método HTTP, ruta, descripción y nivel de autorización), los modelos de datos de Prisma, las reglas de validación, el sistema de email y los módulos auxiliares del backend.

Convenciones generales

- Base URL (producción): <https://reservas.iuce.usal.es>
- Base URL (desarrollo en local): <http://localhost:3000>

Todas las rutas cuelgan de /api/, siguen un estilo REST sobre el App Router de Next.js 14 y responden en JSON.

Códigos de respuesta:

- 200 OK · 201 Creado · 400 Datos inválidos (Zod) · 401 No autenticado
- 403 No autorizado por rol o cuenta suspendida (ban)
- 404 No encontrado · 409 Conflicto (horario, estado o código duplicado)
- 429 Límite de peticiones (rate limit) · 500 Error interno · 503 Servicio degradado (health)

Autenticación y autorización

La autenticación se resuelve con NextAuth.js v5 mediante un provider Credentials propio. En producción se usa autenticación *passwordless* por magic link sobre el correo institucional @usal.es; en desarrollo puede activarse un proveedor mock con NEXT_PUBLIC_USE MOCK_AUTH=true. La sesión es un JWT firmado con duración de 24 horas. El rol (USER, ADMIN, SUPER_ADMIN) se inyecta en el token y la sesión; las rutas privadas se protegen en el middleware.

Tabla 27. Endpoints de autenticación y autorización.

Método	Ruta	Descripción	Auth
POST	/api/auth/magic-link	Genera y envía el enlace de acceso por email. Body: { email }. Solo @usal.es. Máximo 5 tokens activos por email; cada token de un solo uso.	Público
GET·POST	/api/auth/[...nextauth]	Handlers de NextAuth (callback de sesión,	Público

Método	Ruta	Descripción	Auth
		verificación de credenciales).	

Token magic link: 32 bytes aleatorios en hexadecimal, un solo uso (se elimina al consumirse), validez en base de datos de 24 horas.

Salud y monitorización

Tabla 28. *Endpoints de salud y monitorización.*

Método	Ruta	Descripción	Auth
GET	/api/health	Estado del servicio. 200 con status:"ok" si la BD responde; 503 con status:"degraded" si no. Payload: { status, version, timestamp, uptimeSeconds, db: { ok, latencyMs, error }, durationMs }. Sin caché.	Público

Espacios

Consulta del catálogo de espacios. Requiere sesión activa.

Tabla 29. *Endpoints de consulta de espacios.*

Método	Ruta	Descripción	Auth
GET	/api/spaces	Lista los espacios activos (isActive=true), ordenados por nombre. El filtrado por equipamiento, capacidad o edificio se aplica en cliente.	Usuario
GET	/api/spaces/:id	Devuelve un espacio activo por id (404 si no existe o está inactivo).	Usuario

Método	Ruta	Descripción	Auth
GET	/api/spaces/availability	Espacios libres en una franja (query: date, start, end)	Usuario
GET	/api/spaces/:id/occupancy	Intervalos ocupados del espacio para el calendario de disponibilidad	Usuario

Reservas (usuario)

Gestión de las reservas del usuario autenticado.

Tabla 30. *Endpoints de reservas del usuario.*

Método	Ruta	Descripción	Auth
POST	/api/reservations	Crea una reserva o una serie recurrente. Valida rate limit (10/hora), suspensión (403), capacidad, solapamiento (PENDING+APPROVED → 409) y bloqueos (409). Envía emails al usuario y a los administradores.	Usuario
PATCH	/api/reservations/:id	Cancela la reserva propia. Body: { status:"CANCELLED" }. Solo desde PENDING o APPROVED. Envía email de cancelación y registra audit log.	Propietario / Admin

El listado y el detalle de las reservas propias no se exponen como endpoints REST: se renderizan como páginas de servidor (/reservations y /reservations/[id]) con consulta directa a la base de datos.

Recurrencia: patrones WEEKLY, BIWEEKLY y MONTHLY, máximo 52 ocurrencias. Cada instancia se valida de forma individual; las que chocan con otra reserva o bloqueo se omiten y se devuelven en el campo skipped, junto a created, total y recurrenceGroupId.

Perfil de usuario

Endpoints para que el usuario consulte y edite su propia ficha. El email no es editable.

Tabla 31. *Endpoints del perfil de usuario.*

Método	Ruta	Descripción	Auth
GET	/api/users/profile	Devuelve { id, name, email, role, profileCompleted, receiveAdminEmails } del usuario autenticado.	Usuario
PATCH	/api/users/profile	Actualiza el nombre (2–100 caracteres) y marca profileCompleted=true. Body: { name } (y opcionalmente receiveAdminEmails, solo administradores).	Usuario

Notificaciones

Notificaciones derivadas en tiempo de consulta a partir de las reservas (no existe una tabla de notificaciones).

Tabla 32. *Endpoints de notificaciones.*

Método	Ruta	Descripción	Auth
GET	/api/notifications	Admin: solicitudes PENDING (top 10). Usuario: sus reservas revisadas en los últimos 7 días.	Usuario
GET	/api/notifications/count	Contador equivalente (admin: nº de PENDING; usuario: revisadas últimos 7 días).	Usuario

Admin - Reservas

La aprobación y el rechazo se realizan sobre la ruta administrativa. El listado de todas las reservas se sirve como página renderizada en servidor, no como endpoint REST.

Tabla 33. *Endpoints de administración de reservas.*

Método	Ruta	Descripción	Auth
PATCH	/api/admin/reservations/:id	Aprueba o rechaza una solicitud. Body: { status: APPROVED REJECTED, adminNotes? }. Valida la transición permitida; al aprobar re-comprueba el solapamiento contra otras APPROVED. Registra audit log y envía email de decisión.	Admin
PUT	/api/admin/reservations/:id	Modifica una reserva (horario/espacio); notifica al usuario por email	Admin

Admin - Espacios

Alta y mantenimiento del catálogo. Requiere rol ADMIN o SUPER_ADMIN (no existe un GET de listado en /api/admin/spaces: el listado es una página de servidor).

Tabla 34. *Endpoints de administración de espacios.*

Método	Ruta	Descripción	Auth
POST	/api/admin/spaces	Crea un espacio. 201 al crear; 409 si el código ya existe. Registra SPACE_CREATED.	Admin
GET	/api/admin/spaces/:id	Detalle del espacio con el equipamiento deserializado (para el formulario de edición).	Admin
PATCH	/api/admin/spaces/:id	Actualiza campos del espacio. 409 si el nuevo código colisiona. Registra SPACE_UPDATED.	Admin
DELETE	/api/admin/spaces/:id	Soft delete: marca isActive=false (conserva las reservas históricas). 409 si ya estaba inactivo. Registra SPACE_DEACTIVATED.	Admin

Admin – Usuarios

Tabla 35. *Endpoints de administración de usuarios.*

Método	Ruta	Descripción	Auth
GET	/api/admin/users	Lista todos los usuarios con rol, estado de suspensión y último login.	Admin
PATCH	/api/admin/users/:id	Cambia el rol { role } o el estado de suspensión { isBanned, banReason? }. Solo un SUPER_ADMIN modifica a otro SUPER_ADMIN; un SUPER_ADMIN no puede ser suspendido. Registra USER_ROLE_CHANGED / USER_BANNED / USER_UNBANNED.	Admin
POST	/api/admin/users/sync-names	Sincroniza los nombres contra el directorio institucional de la USAL. Solo actualiza cuando difiere; registra	Admin

Método	Ruta	Descripción	Auth
		USER_NAME_CHANGED (source=usal-directory). Devuelve { total, updated, notFound, errors, skipped, details }.	

Admin - Bloqueos

Bloqueos administrativos de franjas. La lectura está abierta a cualquier usuario autenticado; crear y eliminar requiere rol administrativo.

Tabla 36. *Endpoints de administración de bloqueos.*

Método	Ruta	Descripción	Auth
GET	/api/admin/blocked-slots	Lista los bloqueos cuya fecha de fin es futura. Query opcional: spaceId.	Usuario
POST	/api/admin/blocked-slots	Crea un bloqueo. Si hay reservas APPROVED solapadas NO impide la creación: crea el bloqueo y devuelve un campo warnings. Registra BLOCKED_SLOT_CREATED.	Admin
DELETE	/api/admin/blocked-slots?id=	Elimina el bloqueo indicado por id. Registra BLOCKED_SLOT_DELETED.	Admin

Admin - Configuración del sitio

Almacén clave-valor para contenido editable (por ejemplo, las normas de uso).

Tabla 37. *Endpoints de configuración del sitio.*

Método	Ruta	Descripción	Auth
GET	/api/admin/settings?key=	Lee una configuración por clave. Devuelve { key, value }, con value=null si no existe.	Usuario
PUT	/api/admin/settings	Crea o actualiza una configuración. Body: { key, value }. Máximo	Admin

Método	Ruta	Descripción	Auth
		500.000 caracteres. Registra SETTING_UPDATED.	

Admin – Estadísticas

Tabla 38. *Endpoints de estadísticas administrativas.*

Método	Ruta	Descripción	Auth
GET	/api/admin/stats	Volcado agregado: overview (total, pending, approved, rejected, cancelled, expired), serie diaria de los últimos 30 días, ranking de espacios, top 5 de usuarios y las 20 últimas entradas de audit log.	Admin

Admin - Exportaciones (CSV)

Descarga de datos en CSV orientado a Excel en español (separador ";", BOM UTF-8 y escapado de campos al estilo RFC 4180). Cada descarga registra EXPORT_CSV en el audit log.

Tabla 39. *Endpoints de exportación de datos en CSV.*

Método	Ruta	Descripción	Auth
GET	/api/admin/exports/reservations	CSV con todas las reservas (espacio, usuario, fechas, estado, revisor, notas).	Admin
GET	/api/admin/exports/users	CSV con todos los usuarios (rol, suspensión, alta, último login).	Admin
GET	/api/admin/exports/audit	CSV con el registro de auditoría completo.	Admin
GET	/api/admin/exports/reservations-calendar	CSV con la vista de calendario de reservas	Admin

Cron - Recordatorios y expiración

Tabla 40. *Endpoints programados (cron) de recordatorios y expiración.*

Método	Ruta	Descripción	Auth
GET	/api/cron/reminders	Marca como EXPIRED las reservas APPROVED ya finalizadas y envía recordatorio por email a las que comienzan en ~24h (ventana $\pm 1h$). Protegido por header Authorization: Bearer CRON_SECRET.	Cron

Modelos de datos

El modelo de datos completo se describe en la sección C.2.6, donde la *Figura 15* recoge todas las entidades, sus campos, relaciones e índices. Se omite aquí su repetición para mantener una única fuente de referencia.

Reglas de validación

- Reservas: título 3–120 caracteres; descripción opcional, máximo 500; asistentes 1–500.
- Horarios: inicio < fin y no se admiten fechas pasadas. No hay restricción de franja horaria fija en servidor.
- Capacidad: los asistentes no pueden superar la capacidad del espacio.
- Conflictos: no se permiten solapamientos (estados PENDING + APPROVED) en el mismo espacio.
- Límite de creación: máximo 10 reservas por hora y usuario (rate limit en memoria), sin excepción por rol.
- Suspensión: un usuario con isBanned=true recibe 403 al intentar crear reservas.
- Recurrencia: WEEKLY, BIWEEKLY o MONTHLY; máximo 52 instancias; validación individual por instancia.
- Espacios: código único 2–40 (letras, números y guiones); nombre 3–120; capacidad 1–1000.

- Magic link: solo @usal.es; token de 32 bytes, un solo uso, validez 24h; máximo 5 tokens activos por email.
- Configuración: el valor de un SiteSetting admite hasta 500.000 caracteres.

Sistema de email

El sistema realiza el envío transaccional de correo con Resend. Las plantillas son HTML propio (template strings con un layout común de marca IUCE), sin dependencias externas de plantillas, y el asunto se prefija siempre con [IUCE]. Se contemplan ocho tipos de email a lo largo del ciclo de vida de una reserva:

el magic link, que entrega el enlace de acceso único; la confirmación de recepción, que se envía al usuario al crear una reserva; el aviso de nueva solicitud, dirigido a los administradores en ese mismo momento; la notificación de decisión, que comunica al usuario la aprobación o el rechazo tras la revisión; el recordatorio 24 horas antes del inicio, generado por la tarea cron diaria; el aviso de modificación administrativa, que informa al usuario cuando un administrador modifica el horario o el espacio de su reserva; el aviso de cancelación, que se remite al usuario cuando él mismo cancela su reserva; y el aviso de anulación, que se envía cuando es un administrador quien anula una reserva ya aprobada.

En producción, el remitente verificado en Resend es reservas.iuce.usal.es (DKIM, SPF y MX gestionados por el CPD de la USAL). En desarrollo, sin RESEND_API_KEY válida (o con el valor re_placeholder), el envío real se omite y la llamada se registra en consola como [DEV] Email no enviado.

Utilidades del backend

Módulos auxiliares que no exponen endpoints pero soportan funcionalidad transversal:

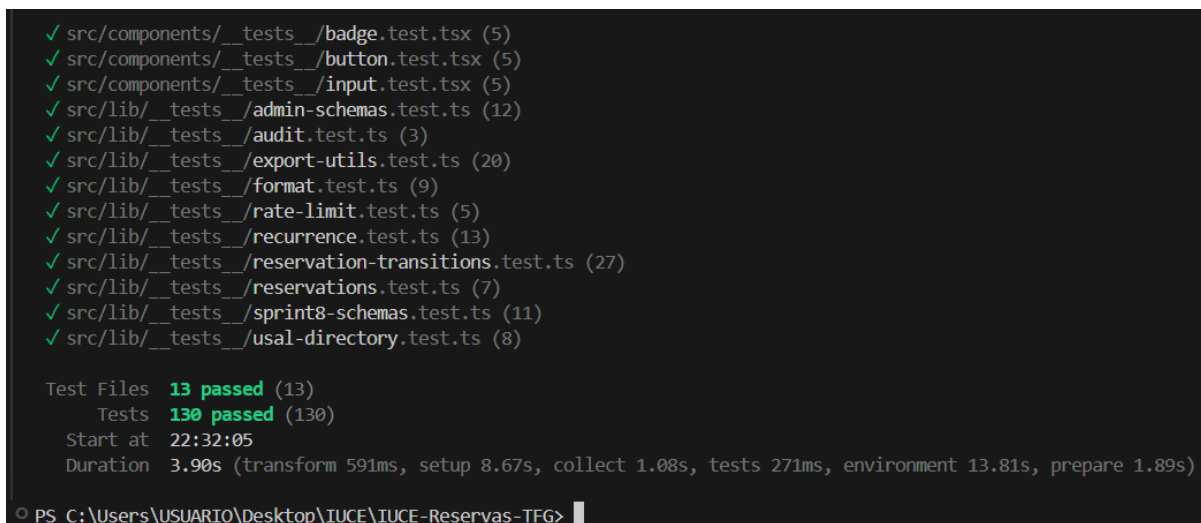
- lib/usal-directory.ts - cliente del directorio institucional de la USAL; resuelve el nombre completo a partir de un correo @usal.es.
- lib/recurrence.ts - expansión de recurrencias a fechas (máx. 52).
- lib/reservations.ts - máquina de estados (canTransitionTo) y comprobación de rol administrativo.
- lib/rate-limit.ts - limitador de peticiones en memoria, por scope.
- lib/export-utils.ts - generación de CSV orientado a Excel en español (separador ";", BOM UTF-8 y escapado al estilo RFC 4180). Solo CSV.
- lib/audit.ts - helper logAudit(entry); captura errores internamente para no abortar la operación principal.
- lib/admin-guard.ts - requireAdmin(): valida sesión y rol antes de cada acción administrativa.
- lib/email.ts · lib/validations.ts (esquemas Zod) · lib/prisma.ts (cliente singleton) · lib/format.ts · lib/space-images.ts.

D.4. Pruebas del sistema

La calidad del sistema se sustenta en una batería de pruebas unitarias y de integración automatizadas, ejecutadas con Vitest. La versión 1.0.0 cuenta con 130 pruebas distribuidas en 13 archivos, todas en verde, que cubren la lógica crítica de negocio.

Las pruebas se concentran en los puntos donde un error tendría mayor impacto: la máquina de transiciones de estado de las reservas (PENDING → APPROVED/REJECTED, cancelación por el propietario, anulación administrativa tras la aprobación y terminalidad de los estados REJECTED, CANCELLED y EXPIRED), los esquemas de validación con Zod, el generador de reservas recurrentes (patrones semanal, quincenal y mensual), la exportación a CSV (separador de punto y coma, marca de orden de bytes UTF-8 y escape de campos para Excel en español) y el control de límite de peticiones.

La batería completa se ejecuta con el comando `npm test` y finaliza en pocos segundos. Además, la integración continua configurada en GitHub Actions lanza, en cada cambio, las fases de análisis estático de estilo, comprobación de tipos de TypeScript y pruebas sobre Node.js 20 con una base de datos PostgreSQL 16 efímera, y añade la compilación únicamente en las integraciones sobre la rama principal, de modo que cualquier fallo bloquea la integración antes de llegar a producción. El análisis estático de SonarCloud completa la verificación con el Quality Gate en verde.



```
✓ src/components/__tests__/badge.test.tsx (5)
✓ src/components/__tests__/button.test.tsx (5)
✓ src/components/__tests__/input.test.tsx (5)
✓ src/lib/__tests__/admin-schemas.test.ts (12)
✓ src/lib/__tests__/audit.test.ts (3)
✓ src/lib/__tests__/export-utils.test.ts (20)
✓ src/lib/__tests__/format.test.ts (9)
✓ src/lib/__tests__/rate-limit.test.ts (5)
✓ src/lib/__tests__/recurrence.test.ts (13)
✓ src/lib/__tests__/reservation-transitions.test.ts (27)
✓ src/lib/__tests__/reservations.test.ts (7)
✓ src/lib/__tests__/sprint8-schemas.test.ts (11)
✓ src/lib/__tests__/usal-directory.test.ts (8)

Test Files 13 passed (13)
Tests 130 passed (130)
Start at 22:32:05
Duration 3.90s (transform 591ms, setup 8.67s, collect 1.08s, tests 271ms, environment 13.81s, prepare 1.89s)

PS C:\Users\USUARIO\Desktop\IUCE\IUCE-Reservas-TFG>
```

Figura 23. Ejecución de la suite de pruebas (130 tests en verde con Vitest).

Overview

No tags • 12k Lines of Code • Last analysis 35 minutes ago

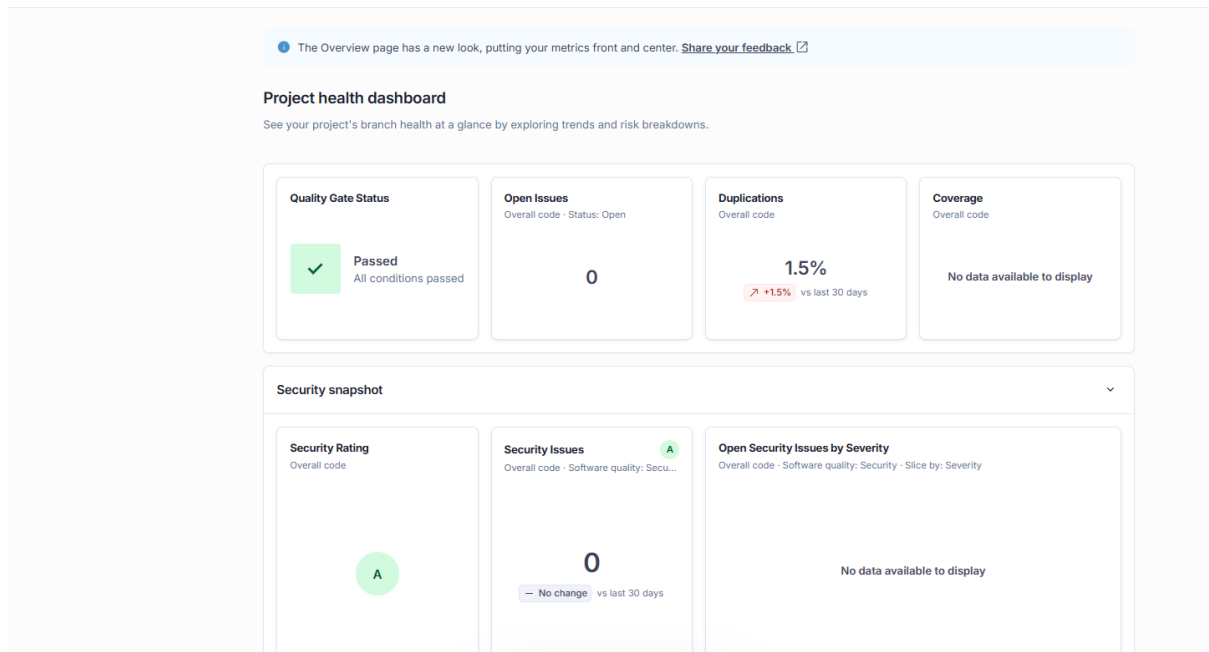


Figura 24. *Panel de SonarCloud con el Quality Gate superado.*